

spath3 宏包：文档

Andrew Stacey

loopspace@mathforge.org

v2.8 发布于 2024/05/31

目录

1	引言	1
2	TikZ 键	2
2.1	路径名称	2
2.2	保存和使用软路径	3
2.3	变换例程	5
2.4	交点例程	6
2.5	使用组件	7
2.6	缩短和分割路径	10
2.7	导出路径	11
2.8	结	11
2.9	坐标	11
3	示例	12

1 引言

`spath3` 宏包最初设计为一个低级宏包，用于操作 PGF/TikZ 定义的 软路径。软路径是作者在 $\text{\LaTeX}$ 文档中的 `tikzpicture` 环境中编写的内容与最终写入输出文件之间翻译堆栈的一个阶段。大部分复杂的处理在软路径构建时已经完成，但它仍然是一个非常明确的 $\text{\TeX}$ 对象，并且还没有考虑最终的输出文件格式（例如 PDF、DVI 或 SVG）。因此，它非常适合在此阶段进行修改，本宏包提供了一组用于此目的的例程。

最初的目的是提供一个通用核心,在此基础上构建其他宏包。事实上, `calligraphy`、`knots` 和 `tilings` 宏包都使用了本宏包。然而,随着时间的推移,我发现自己希望在更高级别使用本宏包的例程,因此设计了一些用户级接口。本文档将对这些接口进行说明。

为了澄清本文档(以及更一般地,本宏包)中使用的一些术语,我将路径视为由 `线段` 和 `组件` 组成。`线段` 是最小的绘图单元。因此,它可能是一条直线或一条贝塞尔曲线。`组件` 是路径的最小连续部分。因此,每个组件都以一个移动命令开始,并持续到下一个移动命令。为了便于实现(并为了在书法宏包中启用铜版笔!),一个独立的移动被视为一个组件。

本宏包无疑存在错误,也有我尚未实现的有用功能。如果您发现了其中任何一种,请告诉我!最好的方法是在 GitHub 上的代码仓库中提交问题,网址是 <https://github.com/loopspace/spath3>。

2 TikZ 键

```
\usetikzlibrary{spath3}
```

`spath3` TikZ 库定义了一组键,可用于修改软路径。这些键都定义在 `spath` 家族中,因此所有以下键都应以 `spath/` 为前缀,或者事先使用键 `spath/.cd` (但请注意,我尚未实现将未知键发送回主 `tikz` 目录)。

如果路径不存在或为空,这些键会尝试优雅地失败¹。目的是文档仍然应该在日志文件(和控制台输出)中编译并显示警告。如果未发生这种情况,请报告。

2.1 路径名称

软路径存储在宏中,但通过名称引用。宏由名称构建,但可以有前缀和后缀。默认情况下,这些设置与 `intersections` 库兼容——本宏包和该库将它们的路径存储在相同的底层宏中。一些在此宏包之上构建的宏包(例如 `knot` 和 `penrose` 宏包)安装它们自己的前缀和后缀以避免冲突。

```
set prefix set prefix={<前缀>}
set suffix set suffix={<后缀>}
prefix      prefix=[<名称>]
suffix      suffix=[<名称>]


---

set name=[<名称>]
```

前两个键直接设置前缀和后缀。其他键允许为前缀和后缀定义标签,以便在不同前缀和后缀之间快速切换。默认值(使用键 `default` 或未指定键)与 `intersections` 库匹配。`knots` 库使用此功能设置不同的前缀。

¹在组内定义的路径,如果在组外引用,将为空而不是不存在。

直接使用名称而不是宏也使得将软路径与一组 TikZ 样式关联成为可能（这在将路径分解为组件时特别有用）。

```
current \path (0,0) -- (3,0) to[out=90, in=90] (1,0) [spath/adjust and close=current];
```

有一个特殊的路径名称，`current`，它指当前正在构建的路径。需要明确的是，对此名称的测试在应用任何前缀或后缀之前进行，因此它仅取决于用户使用什么。

当遇到此名称时，当前路径被复制到一个宏中，并且该路径在命令中使用。如果命令可能以某种方式修改路径，那么在命令完成后，旧的当前路径将被新的路径替换。有几个命令接受多个路径，`current` 可以用作任何路径的替代，但通常只有一个路径被修改，因此只有该路径会用于替换旧的当前路径。

它只在路径构建上下文中才有意义，并且它指的是调用时刻的路径，因此它在路径上的主要选项中没有太多作用，但它可以在后续处理中的选项中使用。

有时无法在路径中间放置命令（例如，如果路径命令埋藏在另一个宏中），因此有一个键可以用于将事情延迟到构建结束：

```
end path construction={code}
```

这会在路径构建完成后立即调用 `code`。

2.2 保存和使用软路径

```
save      save=<名称>
save global save global=<名称>
```

使用名称 `<名称>` 保存当前路径。这会延迟到路径完全构建后才执行，因此可以在主命令的选项中发出。

以这种方式构建的软路径是局部于发出路径命令的组的。`global` 版本全局保存路径，当原始路径位于作用域甚至另一个 `tikzpicture` 内部时很有用。

如果想保存节点的边框，由于存在额外的分组级别，必须使用此键的全局版本。

```
clone      clone={<目标>}{<源>}
clone globally clone globally={<目标>}{<源>}
```

将一个软路径克隆到另一个。在第二个中，克隆是全局的（原始路径不必是）。

通过使用 `current` 作为源名称，当前软路径的当前状态被复制。这可以用来绕过 `save path` 必须等到路径完全构建后才能保存的限制。

<code>show</code>	<code>show=<名称></code>
<code>show current path</code>	<code>show current path</code>

这些键在终端和日志文件中显示路径。它们进行了一些合理的行格式化，以便路径相对容易扫描。

键 `show current path` 延迟日志记录直到路径完全构建。要在构建的中间阶段查看路径，请使用 `show=current`。

<code>use</code>	<code>use=<名称></code>
	<code>use={<名称, 选项>}</code>

这在路径声明的当前连接点使用先前保存的软路径。如果在路径开始之前，它是新路径的初始部分。如果在路径构建期间，它就会被插入到当前位置。请注意，任何直接影响软路径的键都应在此键之前应用。

路径可以首先通过使用第二种形式进行修改——请注意，就 `use` 键而言，整个内容是一个单一参数。选项是一个逗号分隔列表，可以包括：

- `reverse` 首先反转插入的路径。
- `weld`、`no weld` 决定是否将插入的路径焊接到当前路径。焊接意味着移除插入路径开头的 `move to`。请注意，这不会移动插入的路径，因此这通常会修改插入路径的第一个线段，可能以意想不到的方式。预期的用途是，当使用 `weld` 键时，插入的路径从当前路径结束的地方开始，无论是它已经如此，还是因为 `move` 键也已被调用。但没有什么可以阻止任何人在其他情况下使用此键。
- `move`、`no move` 决定是否平移插入的路径，使其从当前路径结束的地方开始。
- `transform={<变换>}` 将指定的变换应用于插入的路径。请参阅第 2.3 节中的 `transform` 键。
- `use current transformation` 将当前变换应用于插入的路径。请注意，由于键是先处理后执行的，此键始终在 `transform` 键之前应用。
- 任何其他键都作为路径的名称（因此不必首先指定），以最后一个为准。

关于变换有一点需要注意。在软路径构建完成时，所有可用的变换都已应用。这意味着当将软路径重新插入到高级命令（如 `\draw`）中时，现有变换不应产生任何影响（除了通过 `transform` 键指定的那些）。这在节点放置等情况下会变得有点棘手——在节点上使用 `pos=D` 键会将该节点放置在路径上的特定点。参数的解释方式与第 2.9 节中描述的 `spath` 坐标系相同。在恢复路径时，库会尝试正确设置 TikZ 的各种内部结构，但我可能忽略或未考虑某些方面，特别是关于现有变换；如果您发现任何异常情况，请向我报告。

```

restore
restore reverse insert
insert reverse
append
append reverse
append no move
append reverse no move

```

这些都是 `use` 的各种版本的别名（它们最初作为独立代码存在，后来我将它们统一为 `use` 的变体）。等价关系如下：

- `restore` 和 `insert` 是 `use` 的别名。
- `append` 设置 `move` 和 `weld` 键。
- `append no move` 只设置 `weld` 键。
- `reverse` 设置 `reverse` 键。

```

join with join with={< 路径 >}{< 路径, 选项 >}

```

这将第二个路径连接到第一个路径，并应用指定的选项。选项与 `use` 键相同。第一个路径被修改，第二个路径不被修改（第二个路径在应用任何修改之前被复制）。

`join with={current}{...}` 几乎等同于 `use={...}`。区别在于 `use={...}` 应用了一个额外的检查，以确保生成的路径不为空。

```

to to={< 名称 >}

```

这从一个软路径定义了一个 `to` 路径，因此它将软路径插入到当前路径中，以跨越起点和终点之间的间隙。路径通过旋转、平移和均匀缩放进行变换，使其精确地跨越 `to` 语法所需的间隙。（如果软路径的起点和终点非常接近，则它不会跨越间隙。）

2.3 变换例程

以下所有键都对软路径应用某种变换。它们不渲染路径，而只是调整它（但它们可以通过使用路径名 `current` 来修改当前路径）。全局版本全局应用其变换，否则它局部于当前组（或作用域）。

```

reverse reverse={< 名称 >}
reverse global reverse globally={< 名称 >}

```

就地反转软路径。如果您想在同一路径中使用原始路径及其反转（例如，用于构建要填充的区域），请先使用 `clone` 键复制它。

```

translate translate={< 名称 >}{<x-尺寸 >}{<y-尺寸 >}
translate global translate globally={< 名称 >}{<x-尺寸 >}{<y-尺寸 >}

```

按给定尺寸平移软路径。

<code>transform</code>	<code>transform={⟨ 名称 ⟩}{⟨ 变换 ⟩}</code>
<code>transform global</code>	<code>transform globally={⟨ 名称 ⟩}{⟨ 变换 ⟩}</code>

这会将变换应用于软路径。变换由 TikZ 处理，因此应由 TikZ 级别的变换组成，例如 `shift={(2,2)}`。

<code>span</code>	<code>span={⟨ 名称 ⟩}{⟨ 起点 ⟩}{⟨ 终点 ⟩}</code>
<code>span global</code>	<code>span globally={⟨ 名称 ⟩}{⟨ 起点 ⟩}{⟨ 终点 ⟩}</code>

这将命名路径进行变换，使其从起点到终点。与 `to` 路径构造和 `splice` 方法一样，如果路径的起点和终点非常接近，则此方法将不起作用。

2.4 交点例程

要使用这些功能，您需要使用 `intersections` 库。

<code>split at self intersections</code>	<code>split at self intersections=⟨ 路径 ⟩</code>
<code>split globally at self intersections</code>	<code>split globally at self intersections=⟨ 路径 ⟩</code>

这会在命名软路径与其自身相交的点处插入断点。断点不是间隙，要实现间隙，请在此之后使用缩短例程，它们是组件的更改。想象一下，就好像您在那一点将笔从页面上移开，然后又立即放回原位一样。

<code>split at intersections with</code>	<code>split at intersections with={⟨ 第一个 ⟩}{⟨ 第二个 ⟩}</code>
<code>split globally at intersections with</code>	<code>split globally at intersections with={⟨ 第一个 ⟩}{⟨ 第二个 ⟩}</code>

这会在第一个路径与第二个路径相交的地方插入断点。第二个路径不改变。

<code>split at intersections</code>	<code>split at intersections={⟨ 第一个 ⟩}{⟨ 第二个 ⟩}</code>
<code>split globally at intersections</code>	<code>split globally at intersections={⟨ 第一个 ⟩}{⟨ 第二个 ⟩}</code>

这会在一对路径的相互交点处插入断点。

<code>replace lines</code>	<code>replace lines=⟨ 路径 ⟩</code>
<code>replace lines globally</code>	<code>replace lines globally=⟨ 路径 ⟩</code>

PGF 交点例程在处理平行或近乎平行的线时存在困难。一种解决办法是将所有线段替换为“曲线”。这样做是为了使曲线的参数化与它所替换的线段的参数化相匹配。

2.5 使用组件

<code>get components of</code>	<code>get components of={⟨ 路径 ⟩}{⟨ 宏 ⟩}</code>
<code>get components of globally</code>	<code>get components of globally={⟨ 路径 ⟩}{⟨ 宏 ⟩}</code>
<code>\GetComponentOf</code>	<code>\GetComponentOf{⟨ 宏 ⟩}{⟨ 数字 ⟩}</code>

这会将路径分解为其组件列表，这些组件存储在宏中。该宏可以在 `\foreach` 中使用。

该宏由逗号分隔的组件名称列表组成（实际使用的名称形式为 `anonymous_N`）。要访问单个组件，请使用命令 `\GetComponentOf`。这可以直接在任何其他键（例如 `use`）中代替路径名称使用（它只是 `LaTeX3` 命令 `\clist_item:Nn`）。

请注意，这些是原始路径组件的副本。更改组件不会更新原始路径。

<code>render components</code>	<code>render components=⟨ 路径 ⟩</code>
--------------------------------	---------------------------------------

这会将给定路径的组件渲染为单独的 TikZ 命令，以便每个组件都可以单独设置样式。它应用以下样式（按此顺序）：

1. `every spath component`
2. `spath component ⟨number⟩`
3. `spath component=⟨number⟩`
4. `every ⟨path⟩ component`
5. `⟨path⟩ component ⟨number⟩`
6. `⟨path⟩ component=⟨number⟩`

<code>insert gaps after components</code>	<code>insert gaps after components={⟨ 路径 ⟩}{⟨ 间隙 ⟩}{⟨ 组件 ⟩}</code>
<code>insert gaps globally after components</code>	<code>insert gaps after components={⟨ 路径 ⟩}{⟨ 间隙 ⟩}</code>
<code>insert gaps after segments</code>	<code>insert gaps globally after components={⟨ 路径 ⟩}{⟨ 间隙 ⟩}{⟨ 组件 ⟩}</code>
<code>insert gaps globally after segments</code>	<code>insert gaps globally after components={⟨ 路径 ⟩}{⟨ 间隙 ⟩}</code>
	<code>insert gaps after segments={⟨ 路径 ⟩}{⟨ 间隙 ⟩}{⟨ 线段 ⟩}</code>
	<code>insert gaps after segments={⟨ 路径 ⟩}{⟨ 间隙 ⟩}</code>
	<code>insert gaps globally after segments={⟨ 路径 ⟩}{⟨ 间隙 ⟩}{⟨ 线段 ⟩}</code>
	<code>insert gaps globally after segments={⟨ 路径 ⟩}{⟨ 间隙 ⟩}</code>

这通过缩短指定组件的末端和下一个组件的开头来在路径组件之间插入间隙。组件列表通过 `\foreach` 循环传递，因此可以使用 `2,4,...,16` 等语法。如果未给出组件列表，则在所有组件之间插入间隙。

`segments` 版本作用于路径的线段而不是其组件。线段是最小的绘图单元，即直线或贝塞尔曲线。

<code>join components</code>	<code>join components={⟨ 路径 ⟩}{⟨ 组件 ⟩}</code>
<code>join components globally</code>	<code>join components globally={⟨ 路径 ⟩}{⟨ 组件 ⟩}</code>

这会移除给定组件与前一个组件之间的 `move` 命令。组件列表由 `\foreach` 处理。如果组件是第一个组件，则它会连接到最后一个组件。

<code>join components with</code>	<code>join components with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
<code>join components globally with</code>	<code>join components with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 列表 ⟩}</code>
<code>join components upright with</code>	<code>join components upright with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
<code>join components globally upright with</code>	<code>join components upright with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 列表 ⟩}</code>
<code>join components with curve</code>	<code>join components with curve={⟨ 路径 ⟩}</code>
<code>join components globally with curve</code>	<code>join components with curve={⟨ 路径 ⟩}{⟨ 列表 ⟩}</code>

	<code>join components globally with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
	<code>join components globally with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 列表 ⟩}</code>
	<code>join components globally upright with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
	<code>join components globally upright with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 列表 ⟩}</code>
	<code>join components globally with curve={⟨ 路径 ⟩}</code>
	<code>join components globally with curve={⟨ 路径 ⟩}{⟨ 列表 ⟩}</code>

这会将 `splice path` 插入到由逗号分隔列表指定的 `path` 组件之间的间隙中。拼接路径插入到每个指定组件和下一个组件之间。如果未给出 `{⟨list⟩}` (或为空)，则拼接路径插入到每个组件之间，但首先执行 `点焊` 以连接任何一个组件的末端是下一个组件的起点的地方。点焊仅在组件列表为空时执行。

这 不会 关闭生成的路径，即使列表中指定了最后一个组件，有关此内容请参阅键 `close with`。

请注意，由于此键有一个可选的第三个参数，因此第二个参数必须始终用大括号括起来，除非它是一个单一标记。

`upright` 版本会进行一个小测试，以查看间隙是否倒置，如果是，则插入拼接路径的反射。

`with curve` 版本在间隙中插入一个单一的贝塞尔曲线。曲线的选择与间隙两侧线段的切线匹配，并使用 John Hobby 的贝塞尔曲线构造（如 `hobby TikZ` 库中所用）。

<code>spot weld</code>	<code>spot weld=⟨ 路径 ⟩</code>
<code>spot weld globally</code>	<code>spot weld globally=⟨ 路径 ⟩</code>

这会移除路径中任何两个组件之间，当一个组件的终点与下一个组件的起点相同时的 `move` 命令（这里的误差容限是 0.01pt）。

<code>remove empty components</code>	<code>remove empty components=⟨ 路径 ⟩</code>
<code>remove empty components globally</code>	<code>remove empty components globally=⟨ 路径 ⟩</code>

这会移除路径中的空组件（仅包含一个移动命令的组件）。

<code>remove components</code>	<code>remove components={⟨ 路径 ⟩}{⟨ 列表 ⟩}</code>
<code>remove components globally</code>	<code>remove components globally={⟨ 路径 ⟩}{⟨ 列表 ⟩}</code>

这会移除路径中列出的组件。与其他列表例程一样，列表首先通过 `foreach` 解析。

<code>open</code>	<code>open={⟨ 路径 ⟩}</code>
<code>open globally</code>	<code>open globally={⟨ 路径 ⟩}</code>

这些通过移除所有闭合元素来打开路径。如果闭合元素实质性地增加了路径，则它们将被线段替换。

<code>close</code>	<code>close={⟨ 路径 ⟩}</code>
<code>close globally</code>	<code>close globally={⟨ 路径 ⟩}</code>
<code>close with</code>	<code>close with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
<code>close globally with</code>	<code>close globally with={⟨ 路径 ⟩}{⟨ 拼接路径 ⟩}</code>
<code>close with curve</code>	<code>close with curve={⟨ 路径 ⟩}</code>
<code>close globally with curve</code>	<code>close globally with curve={⟨ 路径 ⟩}</code>
<code>adjust and close</code>	<code>adjust and close={⟨ 路径 ⟩}</code>
<code>adjust and close globally</code>	<code>adjust and close globally={⟨ 路径 ⟩}</code>

这些都关闭给定路径的最后一个组件。前两个如果组件的起点和终点不够接近，则会插入一条线段。两个 `with` 变体允许您指定另一条路径插入到路径的末端和起点之间。两个 `with curve` 变体在间隙中插入一条贝塞尔曲线；曲线的选择取决于连接路径段的切线，并使用 John Hobby 的贝塞尔曲线构造（如 `hobby TikZ` 库中所用）。两个 `adjust` 变体移动路径的终点，使其与该组件的起点位于同一位置。如果最后一个线段是贝塞尔曲线，则第二个控制点也会移动。（使用 `adjust` 键的目的是当组件的起点和终点非常接近时，调整会产生比添加一条线更小的视觉效果——参见示例 6。）

<code>splice</code>	<code>splice={⟨ 初始路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 最终路径 ⟩}</code>
<code>splice global</code>	<code>splice globally={⟨ 初始路径 ⟩}{⟨ 拼接路径 ⟩}{⟨ 最终路径 ⟩}</code>

这将中间路径拼接在初始路径和最终路径之间的间隙中。中间路径被变换以适应（不要尝试对起点和终点彼此靠近的路径这样做），并且路径被连接，使得初始路径的最后一个组件和拼接路径的第一个组件成为单个组件，另一端也类似。

2.6 缩短和分割路径

<code>shorten at end</code>	<code>shorten at start={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>
<code>shorten at start</code>	<code>shorten at end={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>
<code>shorten at both ends</code>	<code>shorten at both ends={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>
<code>shorten globally at end</code>	<code>shorten globally at start={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>
<code>shorten globally at start</code>	<code>shorten globally at end={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>
<code>shorten globally at both ends</code>	<code>shorten globally at both ends={⟨ 路径 ⟩}{⟨ 长度 ⟩}</code>

这会将路径从指定末端缩短给定长度。缩短操作确保其沿着原始路径，但因此长度不能完全保证精确。对于贝塞尔路径和末端非常短的线段尤其如此。

它使用末端的导数来计算缩短量。如果想大幅缩短，最好多次少量缩短。

<code>split at</code>	<code>split at={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split globally at</code>	<code>split globally at={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split at into</code>	<code>split at into={⟨ 起始路径 ⟩}{⟨ 结束路径 ⟩}{⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split globally at into</code>	<code>split globally at into={⟨ 起始路径 ⟩}{⟨ 结束路径 ⟩}{⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split at keep start</code>	<code>split at keep start={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split globally at keep start</code>	<code>split globally at keep start={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split at keep end</code>	<code>split at keep end={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split globally at keep end</code>	<code>split globally at keep end={⟨ 路径 ⟩}{⟨ 参数 ⟩}</code>
<code>split at keep middle</code>	<code>split at keep middle={⟨ 路径 ⟩}{⟨ 参数 ⟩}{⟨ 参数 ⟩}</code>
<code>split globally at keep middle</code>	<code>split globally at keep middle={⟨ 路径 ⟩}{⟨ 参数 ⟩}{⟨ 参数 ⟩}</code>

这些键在由参数指定的点处分割路径。参数的解释在第 2.9 节中描述。

前两个键用分割路径替换原始路径，因此在由参数指定的点处引入了一个断点。后两个键将分割路径保存为新路径。

名称中带有 `keep` 的键会分割路径并保留指定的部分（因此丢弃其余部分）。保留中间部分的键需要两个参数来指定断点。

<code>arrow shortening</code>	<code>arrow shortening</code>
	<code>arrow shortening={⟨ true/false ⟩}</code>

当箭头添加到路径时，路径会被缩短，以便箭头尖端位于路径应终止的位置。因此，放置箭头有两个效果：修改路径和渲染箭头。前者应在软路径操作之前发生，后者应在之后发生。此键允许进行必要的分离。在路径使用时调用它会禁用该连接点的缩短。

请注意，如果路径（或其最后一个线段）非常短，那么当箭头最终放置时，它可能会反向指向。修复此问题已列入 待办事项 列表！

2.7 导出路径

有两个键用于导出路径。

```
save to aux save to aux=< 路径 >
```

这会将路径保存到辅助文件，以便在下次运行时再次可用。

```
export to svg export to svg=< 路径 >
```

将路径保存到文件 `path.svg` 作为 SVG 文档。

2.8 结

```
knot knot={< 路径 >}{< 间隙 >}{< 组件 >}  
global knot global knot={< 路径 >}{< 间隙 >}{< 组件 >}  
draft mode
```

这种样式结合了上述各种功能,使其更容易绘制结和链以及类似的图。它扩展为:

```
knot/.style n args={3}{  
  spath/.cd,  
  split at self intersections=#1,  
  insert gaps after components={#1}{#2}{#3},  
  maybe spot weld=#1,  
  render components=#1  
}
```

(`global` 版本使路径操作命令全局生效。)

这会在路径自相交的点处分割路径,然后在指定组件之间插入间隙。键 `maybe spot weld` 执行 `spot weld`, 具体取决于键 `draft mode` 是否设置为 `true` 或 `false`。这里的重点是,在设计结时,不将组件焊接在一起很有用,因为这会改变组件计数。但是一旦在所需位置插入了间隙,焊接剩余的组件会产生更漂亮的图。

组件可以使用 `render components` 中描述的键进行样式设置。

2.9 坐标

软路径不是自然的 TikZ 对象,因此当它们被替换回 TikZ 路径构造时,它们不能完全与其他 TikZ 对象(例如在路径上的点放置节点)交互(尽管我已尽力使其工作)。为了使事情更容易一些,定义了一个坐标系,它标识了沿软路径特定位置的点,以及应用变换的键。

`spath cs` (`spath cs:{<名称>}{<参数>}`)

位置规范有点技术性。它被指定为从 0 到 1 的数字，但参数的工作方式如下。令 n 为路径上的 线段 数量（这些是构成路径的单个绘图元素）。从 $\frac{k-1}{n}$ 到 $\frac{k}{n}$ 的区间分配给路径的第 k 个线段。然后对于该区间中的参数，位置使用该线段的自然参数化。对于直线，它只是沿线的比例位置，但对于贝塞尔曲线，它使用贝塞尔参数化。

空格至关重要，因此如果 `name` 包含在宏中，则必须以某种方式插入空格。一种选择是将宏括在大括号中，如下一节的示例所示。

<code>transform to</code>	<code>transform to={<路径>}{<参数>}</code>
<code>upright transform to</code>	<code>upright transform to={<路径>}{<参数>}</code>

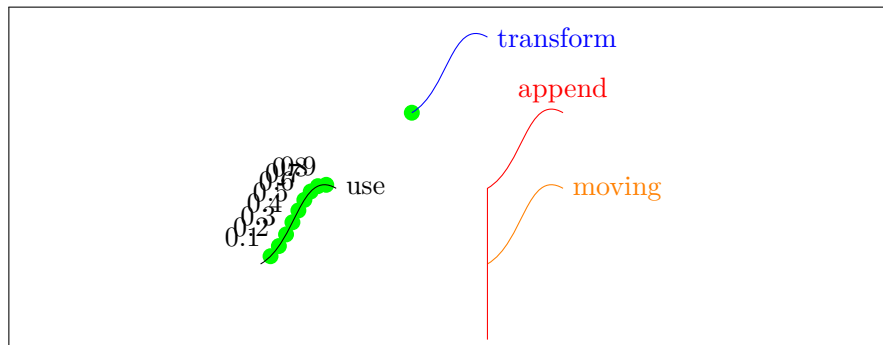
这些键（属于 `spath` 家族）设置变换，使原点位于曲线的指定点（如上所述），并且 x 轴与曲线相切。变换是 正交的，即通过旋转和平移实现。

第一个键对齐轴，使 x 轴沿路径构建时的前进方向。第二个键对齐轴，使 y 轴向上指向页面。第二个键的目的是使其类似于当节点放置在曲线上时 `sloped` 键发生的情况。

3 示例

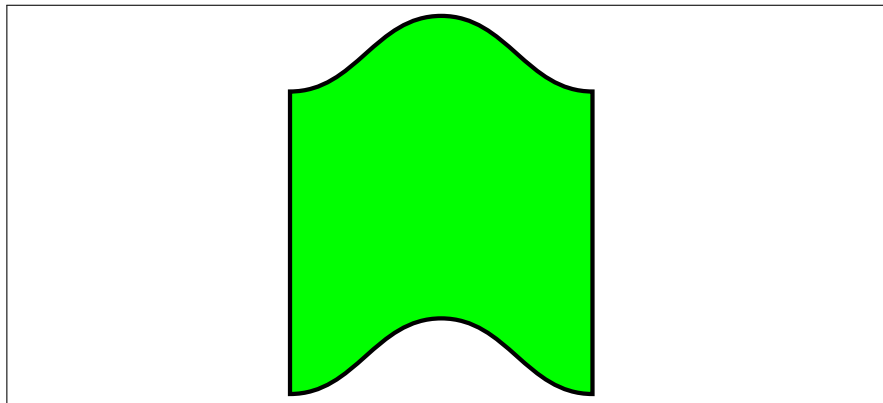
1. 保存和重用。

```
\begin{tikzpicture}
\path[spath/save=rpath] (0,0) to[out=30,in=150] (1,1);
\foreach \k in {1,...,9} {
  \fill[green] (spath cs:rpath 0.\k) circle[radius=3pt];
  \node[above left] at (spath cs:rpath 0.\k) {\(0.\k\)};
}
\fill[green] (2,2) circle[radius=3pt];
\draw[blue, spath/transform={rpath}{shift={(2,2)}}],
      spath/use=rpath] node[right] {transform};
\draw[orange] (3,0) [spath/use={rpath, move}] node[right] {moving};
\draw[red] (3,-1) -- +(0,2) [spath/append=rpath] node[above]
      {append};
\draw[spath/use=rpath] node[right] {use};
\end{tikzpicture}
```



2. 反转。

```
\begin{tikzpicture}
\path[spath/save=apath] (0,0) to[out=0,in=180] (2,1)
    to[out=0,in=180] (4,0);
\filldraw[
    green,
    draw=black,
    ultra thick,
    spath/use=apath
] -- ++(0,-4) [spath/use={apath, reverse, move, weld}] -- cycle;
\end{tikzpicture}
```

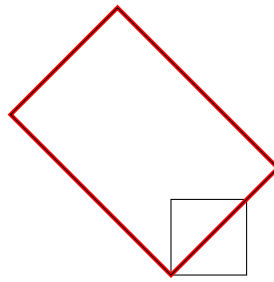


3. 变换。

```

\begin{tikzpicture}
\draw[spath/save=tpath] (0,0) rectangle +(1,1);
\draw[rotate=45, xscale=2, yscale=3, ultra thick, red] (0,0)
    rectangle +(1,1);
\draw[
    spath/transform={tpath}{rotate=45, xscale=2, yscale=3},
    spath/use={tpath}];
\end{tikzpicture}

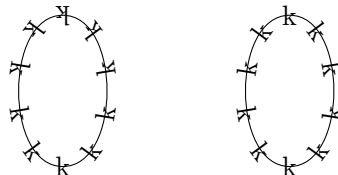
```



```

\begin{tikzpicture}
\draw[spath/save=oval] (0,0) to[out=0,in=0] (0,2)
    to[out=180,in=180] (0,0);
\foreach \k in {0,...,9} {
    \node[transform shape, spath/transform to={oval}{0.\k}] {\k};
}
\begin{scope}[xshift = 3cm]
\draw[spath/save=soval] (0,0) to[out=0,in=0] (0,2)
    to[out=180,in=180] (0,0);
\foreach \k in {0,...,9} {
    \node[transform shape, spath/upright transform to={soval}{0.\k}]
        {\k};
}
\end{scope}
\end{tikzpicture}

```



```

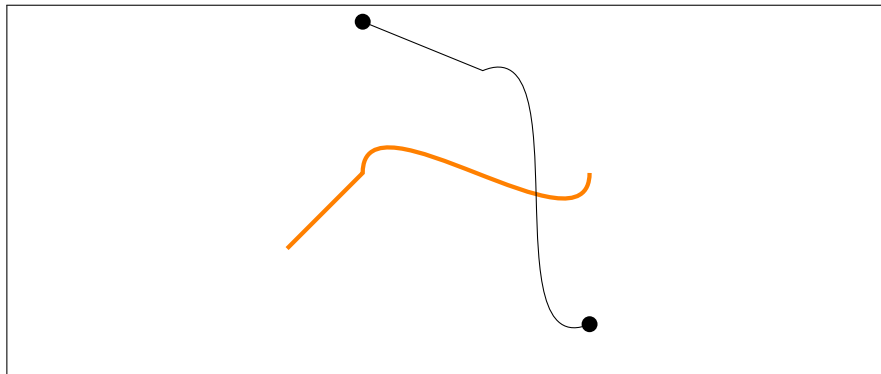
\begin{tikzpicture}
\path[draw=orange,ultra thick,spath/save=a] (3,2) -- ++(1,1)
      to[out=90,in=-90] ++(3,0);

\tikzset{
  spath/span={a}{(4,5)}{(7,1)}
}

\fill
(4,5) circle[radius=3pt]
(7,1) circle[radius=3pt]
;

\draw[spath/use=a];
\end{tikzpicture}

```



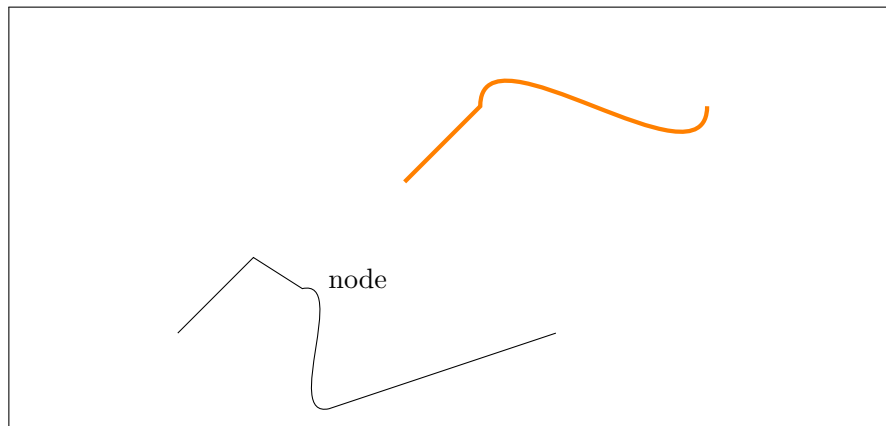
4. To 路径。

```

\begin{tikzpicture}
\path[draw=orange,ultra thick,spath/save=a] (3,2) -- ++(1,1)
      to[out=90,in=-90] ++(3,0);

\draw (0,0) -- ++(1,1) to[spath/to={a}] node[pos=.6,auto] {node}
      ++(2,-1) -- ++(3,1);
\end{tikzpicture}

```

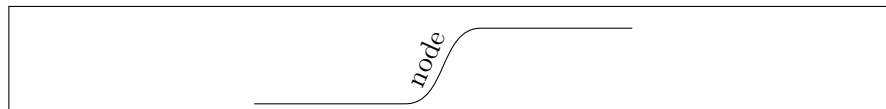


5. 节点放置。

```
\begin{tikzpicture}
\path[spath/save=curve] (0,0) to[out=0,in=180] +(1,1);

\draw (0,0) -- (2,0) [spath/append=curve] node[pos=.5,auto,sloped]
{node} -- +(2,0);

\end{tikzpicture}
```



6. 关闭和调整路径。


```

\begin{tikzpicture}
\draw[line width=.2cm] (0,0) -- +(1,0) -- +(1,1) -- +(0,1) --
    +(.01,.01);

\draw[line width=.2cm] (2,0) -- +(1,0) -- +(1,1) -- +(0,1) --
    +(.01,.01)
-- cycle;

\draw[line width=.2cm] (4,0) -- +(1,0) -- +(1,1) -- +(0,1) --
    +(.01,.01)
[spath/close=current];

\draw[line width=.2cm] (6,0) -- +(1,0) -- +(1,1) -- +(0,1) --
    +(.01,.01)
[spath/adjust and close=current];
\end{tikzpicture}

```



7. 缩短。

```

\begin{tikzpicture}
\path[spath/save=apath] (0,0) foreach \k in {1,...,4} { -- ++(1,0)
    +(0,0)};
\draw[
    ultra thick,
    red,
    spath/.cd,
    shorten at end={apath}{7pt},
    shorten at start={apath}{9pt},
    translate={apath}{0pt}{1pt},
    use=apath,
];
\draw (0,0) circle[radius=9pt] [spath/use=apath] circle[radius=7pt];
\end{tikzpicture}

```



```

\begin{tikzpicture}
\path[spath/save=apath] (0,0) foreach \k in {1,...,4} {
  to[out=0,in=180] ++(1,0) +(0,0)};
\draw[
  ultra thick,
  red,
  spath/.cd,
  shorten at end={apath}{7pt},
  shorten at start={apath}{9pt},
  translate={apath}{0pt}{1pt},
  use=apath,
];
\draw (0,0) circle[radius=9pt] [spath/use=apath] circle[radius=7pt];
\end{tikzpicture}

```



```

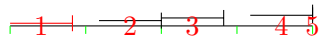
\begin{tikzpicture}
\draw[spath/save=npath] (0,0) foreach \k in {1,...,4} { -- ++(1,0)
    +(0,0)};
\draw[green] (0,0) -- +(0,-3pt) foreach \k in {1,...,4} { --
    +(0,-3pt) ++(1,0)} -- +(0,-3pt);

\tikzset{
    spath/.cd,
    insert gaps after components={npath}{10pt}{1,3},
    get components of={npath}\components,
}

\tikzset{
    path 1/.style={
        red,
    },
}

\foreach[count=\k] \cpt in \components {
    \path[
        draw,
        path \k/.try,
        spath/.cd,
        translate=\cpt{0pt}{\k pt},
        use=\cpt,
    ] +(0,3pt) -- +(0,-3pt);
    \node[text=red] at (spath cs:{\cpt} .5) {\(\k\)};
}
\end{tikzpicture}

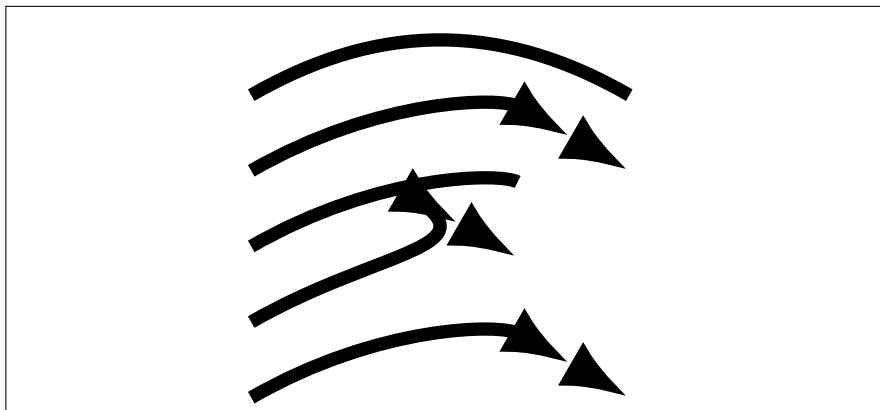
```



```

\begin{tikzpicture}[>=Latex, line width=5pt]
% Just a simple line
\draw (0,1) to[bend left] +(5,0);
% Same line but with arrows, also save the path
\draw[->.>,spath/save=arrow] (0,0) to[bend left] +(5,0);
% Let's redraw that path without the arrows - it's short! But also
  distorted
\draw[spath/use={arrow,transform={yshift=-1cm}}];
% So if we redraw it with arrows it gets doubly shortened
\draw[->.>,spath/use={arrow,transform={yshift=-2cm}}];
% If we disable the shortening, the arrows end up in the right place
\draw[->.>,spath/use={arrow,transform={yshift=-3cm}}, spath/arrow
  shortening=false];
\end{tikzpicture}

```



8. 交点。

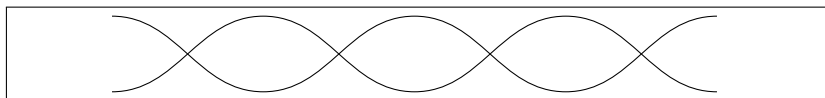
实现交点例程的主要动机之一是提供一种绘制结、链和类似图的不同方法。

(a) 定义辫子的两条路径（通常这些路径将用 `\path` 定义）。

```

\begin{tikzpicture}[
  use Hobby shortcut,
]
\draw[spath/save global=pathA] (0,0) to[out=0,in=180] ++(2,1)
  to[out=0,in=180] ++(2,-1) to[out=0,in=180] ++(2,1)
  to[out=0,in=180] ++(2,-1);
\draw[spath/save global=pathB] (0,1) to[out=0,in=180] ++(2,-1)
  to[out=0,in=180] ++(2,1) to[out=0,in=180] ++(2,-1)
  to[out=0,in=180] ++(2,1);
\end{tikzpicture}

```

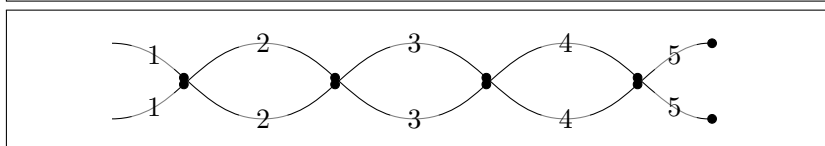


(b) 在路径的相互交点处分割路径，并用组件计数渲染它们。

```
\begin{tikzpicture}
\tikzset{
  spath/.cd,
  split at intersections={pathA}{pathB},
  get components of={pathA}\pathAcomponents,
  get components of={pathB}\pathBcomponents,
}

\foreach[count=\k] \cpt in \pathAcomponents {
  \draw[spath/use=\cpt,-Circle];
  \node[fill=white, fill opacity=.5, circle, text opacity=1] at
    (spath cs:\cpt .5) {\(\k\)};
}

\foreach[count=\k] \cpt in \pathBcomponents {
  \draw[spath/use=\cpt,-Circle];
  \node[fill=white, fill opacity=.5, circle, text opacity=1] at
    (spath cs:\cpt .5) {\(\k\)};
}
\end{tikzpicture}
```



(c) 现在我们在每条路径的特定组件之后插入间隙，然后渲染这些组件。为了表明间隙是真实的，我们使用带图案的背景。尽管路径是全局定义的，但上一个示例中的分割是局部的，因此我们需要在此示例中重复它。

```

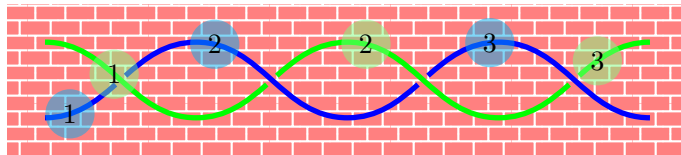
\begin{tikzpicture}
\tikzset{
  spath/.cd,
  split at intersections={pathA}{pathB},
  insert gaps after components={pathA}{5pt}{1,3},
  join components={pathA}{3,5},
  get components of={pathA}\pathAcomponents,
  insert gaps after components={pathB}{5pt}{2,4},
  join components={pathB}{2,4},
  get components of={pathB}\pathBcomponents,
}

\fill[red!50!white] (-.5,-.5) rectangle (8.5,1.5);
\fill[pattern=bricks, pattern color=white] (-.5,-.5) rectangle
(8.5,1.5);

\foreach[count=\k] \cpt in \pathAcomponents {
  \draw[blue, line width=2pt,spath/use=\cpt];
  \node[fill=cyan, fill opacity=.5, circle, text opacity=1] at
    (spath cs:\cpt} .3) {\(\k\)};
}

\foreach[count=\k] \cpt in \pathBcomponents {
  \draw[green, line width=2pt,spath/use=\cpt];
  \node[fill=green!50, fill opacity=.5, circle, text opacity=1]
    at (spath cs:\cpt} .3) {\(\k\)};
}
\end{tikzpicture}

```



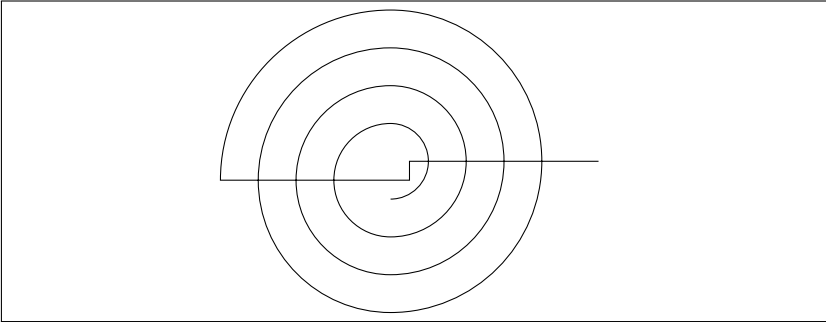
9. 此示例值得注意，因为许多交点是路径线段的末端，表明即使在这种情况下，算法也能很好地工作。

(a) 这是原始路径。

```

\begin{tikzpicture}
\draw[spath/save global=spiral] (5,0) -- (2.5,0) -- ++(0,-.25)
-- ++(-2.5,0)
arc[radius=2.25cm,start angle=180,end angle=90]
arc[radius=2cm,start angle=90, delta angle=-180]
arc[radius=1.75cm,start angle=-90, delta angle=-180]
arc[radius=1.5cm,start angle=90, delta angle=-180]
arc[radius=1.25cm,start angle=-90, delta angle=-180]
arc[radius=1cm,start angle=90, delta angle=-180]
arc[radius=.75cm,start angle=-90, delta angle=-180]
arc[radius=.5cm,start angle=90, delta angle=-180]
;
\end{tikzpicture}

```



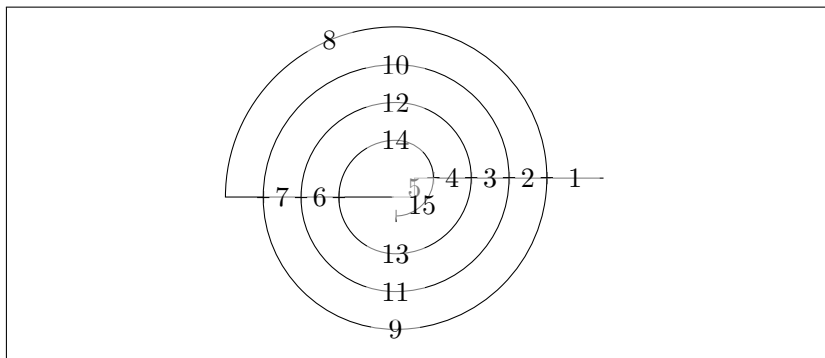
(b) 这在分割后在每个组件上渲染标签。

```

\begin{tikzpicture}
\tikzset{
  spath/.cd,
  split at self intersections=spiral,
  get components of={spiral}\pathcomponents,
}

\foreach[count=\k] \cpt in \pathcomponents {
  \draw[spath/use=\cpt,-|];
  \node[fill=white, fill opacity=.5, circle, text opacity=1] at
    (spath cs:{\cpt} .5) {\(\k\)};
}
\end{tikzpicture}

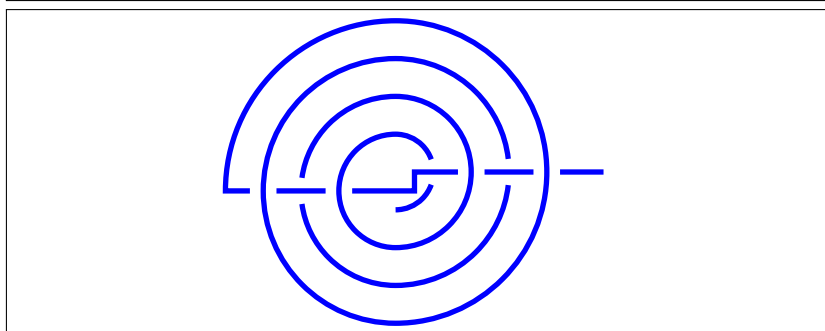
```



(c) 最后，我们将间隙放置在我们想要的位置。

```
\begin{tikzpicture}
\tikzset{
  spath/.cd,
  split at self intersections=spiral,
  insert gaps after components={spiral}{10pt}{1,3,5,7,10,11,14},
  spot weld=spiral,
  get components of={spiral}\pathcomponents,
}

\foreach[count=\k] \cpt in \pathcomponents {
  \draw[blue, line width=2pt,spath/use=\cpt];
}
\end{tikzpicture}
```

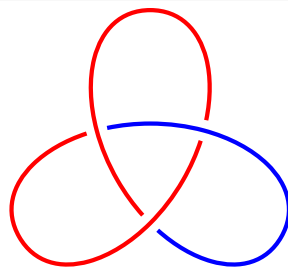


10. 这是一个三叶结，演示了简化创建结的 `knot` 样式。


```

\begin{tikzpicture}[
  use Hobby shortcut,
  every trefoil component/.style={ultra thick, draw, red},
  trefoil component 1/.style={blue},
]
\path[spath/save=trefoil] ([closed]90:2) foreach \k in {1,...,3} {
  .. (-30+\k*240:.5) .. (90+\k*240:2) } (90:2);
\tikzset{spath/knot={trefoil}{8pt}{1,3,5}}
\end{tikzpicture}

```



11. 这是如何用“桥”标记路径的交点。

```

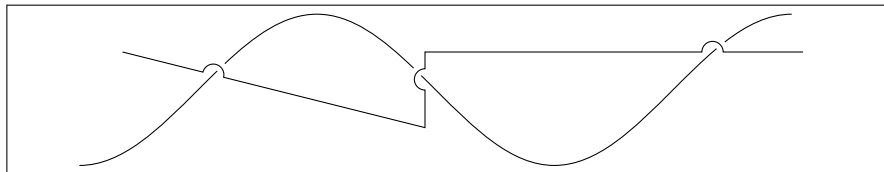
\begin{tikzpicture}
\coordinate (a) at (-1,0.5);
\coordinate (b) at (8,0.5);
\coordinate (c) at (3,-0.5);
\path[spath/save=sine]
(-1.57,-1)
cos ++(1.57,1)
sin ++(1.57,1)
cos ++(1.57,-1)
sin ++(1.57,-1)
cos ++(1.57,1)
sin ++(1.57,1);
\path[spath/save=over] (a) -- (c) |- (b);

\path[spath/save=arc] (0,0) arc[radius=1cm, start angle=180, delta
angle=-180];

\tikzset{
  spath/split at intersections with={over}{sine},
  spath/insert gaps after components={over}{8pt},
  spath/join components upright with={over}{arc},
  spath/split at intersections with={sine}{over},
  spath/insert gaps after components={sine}{4pt},
}

\draw[spath/use=sine];
\draw[spath/use=over];
\end{tikzpicture}

```



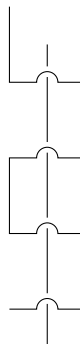
12. 如果像上一个示例那样有很多路径，这里有一个方便的样式可以将它们组合在一起。

```

\tikzset{
  bridging path/.initial=arc,
  bridging span/.initial=8pt,
  bridging gap/.initial=4pt,
  bridge/.style 2 args={
    spath/split at intersections with={#1}{#2},
    spath/insert gaps after
    components={#1}{\pgfkeysvalueof{/tikz/bridging span}},
    spath/join components upright
    with={#1}{\pgfkeysvalueof{/tikz/bridging path}},
    spath/split at intersections with={#2}{#1},
    spath/insert gaps after
    components={#2}{\pgfkeysvalueof{/tikz/bridging gap}},
  }
}

% 如果在序言中使用，这需要用 \AtBeginDocument 括起来
%\AtBeginDocument{%
\tikz[overlay] { \path[spath/save global=arc] (0,0) arc[radius=1cm,
  start angle=180, delta angle=-180]; }%
}%
\begin{tikzpicture}
\path[spath/save=over] (0,0) -| ++(1,1) -| ++(-1,1) -| ++(1,1) -|
  ++(-1,1);
\path[spath/save=under] (.5,-.5) -- ++(0,4);
\tikzset{bridge={over}{under}}
\draw[spath/use=over];
\draw[spath/use=under];
\end{tikzpicture}

```



定义弧的命令确实引入了一个盒子，这可能会引入额外的间距。上面的语法，行

未带有%, 不应该添加空格, 但如果出于某种原因它确实添加了, 那么更复杂的解决方案详见 [如何在不引入伪影的情况下为 spath3 保存或定义 TikZ 路径?](#)。